SoK: Attack and Defense Landscape of Agentic AI Systems

Juhee Kim[1,2]*, Wenbo Guo*[3]*, Dawn Song*[1]
[1] *UC Berkeley,* [2] *Seoul National University,* [3] *UC Santa Barbara*
kimjuhi96@snu.ac.kr, {*dawnsong*}*@berkeley.edu,* {*henrygwb*}*@ucsb.edu*

Abstract

AI agents that combine large language models with non-AI system components are rapidly emerging in real-world applications, offering unprecedented automation and flexibility. However, this unprecedented flexibility introduces complex security challenges that differ from those found in traditional software systems. This paper presents the first comprehensive systematization of knowledge on AI agent security, including an analysis of agents' design space, attack landscape, and defense mechanisms for secure AI agent systems. We further identify open challenges that point to promising directions for future research in this emerging domain. Our work introduces the first systematic framework for understanding the security risks and defense landscapes of AI agents, serving as a foundation for building both secure agentic systems and advancing research in this critical area.

1 Introduction

The rapid advancement of agentic AI systems[1] has fundamentally transformed the AI landscape, marking a paradigm shift from isolated language models to integrated systems that combine Large Language Models (LLMs) with diverse software components. These hybrid systems demonstrate unprecedented capabilities by seamlessly integrating AI models and traditional symbolic software, enabling autonomous task execution and dynamic tool usage. Agentic AI systems span various applications, from software development [28, 44] to web browser use [99, 108].

The security implications of agentic AI systems are becoming increasingly critical as recent incidents demonstrate the severity of potential vulnerabilities. Prompt injection attacks have been exploited to access private GitHub repositories [54], while remote code execution vulnerabilities [88] have enabled unauthorized system access. Data exfiltration attacks [112, 117] have compromised sensitive information through malicious document attachments and email forwarding, servers have exposed user chat and credential data [98, 115], and attackers can gain access to users' personal banking accounts exploiting web agents [13]. These incidents underscore that the flexibility and automation capabilities make agentic systems powerful, while also creating complex security challenges, fundamentally different from traditional software systems or standalone AI models.

Existing literature reviews primarily focus on LLM security [49, 58, 81, 138], and the few recent surveys on agents largely concentrate on specific attack vectors or isolated system components [52, 138, 155]. *The community lacks a systematic view or framework for understanding the overall risks of the agentic system as a whole, as well as guidance for security principles and mechanisms for agentic systems.*

This paper addresses this gap by *providing the first comprehensive systematization of knowledge on the security landscape of agentic AI systems.* We approach agent security from a holistic systems perspective, examining how the combination of LLMs with traditional software creates unique security challenges that cannot be addressed through component-level defenses alone. Our analysis begins with a systematic characterization of agent design dimensions that influence security properties, followed by a comprehensive taxonomy of attack vectors and security risks across the entire agent ecosystem. We then discuss the defense principles for agents and conduct a systematic survey of existing defense mechanisms, categorizing them by their protection approaches. Through the process, we identify limitations in current defenses and discuss open challenges for constructing secure agentic systems. Our contributions are threefold:

- **Agent Design Dimensions:** We present a systematic framework that characterizes agentic AI systems through seven key design dimensions, input trust, access sensitivity, workflow, action, memory, tool, and user interface, and analyze how flexibility along each dimension impacts security risks.
- **Comprehensive Attack Landscape and Taxonomy:**

[*] Work done during the author's visit to UC Berkeley.
[1] In this paper, we use agentic AI systems and AI agents interchangeably.

We develop a systematic taxonomy of attack vectors organized by threat models (i.e., external, user-level, and internal adversaries) and provide a comprehensive classification of nine security risk categories that encompass user interaction, model behavior, memory systems, tool integration, and environment interaction.

- **Defense Landscape Systematization:** We systematically survey existing defense mechanisms in three categories: runtime protection, secure-by-design, and component hardening, identifying specific design dimensions for defense and open challenges.

To the best of our knowledge, this is the first work to systematically analyze the security landscape of agentic AI systems from a comprehensive system perspective. Our systematization provides a foundational framework for understanding agent security risks and defense strategies, guiding future research toward building secure agentic systems. This work can serve as a handbook for researchers and developers working with agentic AI systems.

2 Overview

In this section, we define the scope of our work, introduce our methodology, and discuss the key differences from existing agent security surveys.

Scope. We focus on security risks and defenses that are unique to, or significantly amplified in, agentic systems compared to traditional software and standalone LLMs. We first characterize how agents differ from standalone models across seven design dimensions (§3). We then analyze agent-specific security risks both at the component and system levels (§4). For risks that exist in non-agentic LLMs (e.g., jailbreak, hallucination), we emphasize how agent autonomy and environment access magnify their impact (e.g., data exfiltration, unintended system manipulation). We exclude attacks targeting model internals, such as model inversion [38], as these do not fundamentally worsen in agentic contexts where agents operate purely through inference-time input-output interfaces. Finally, §5 systematizes the agent defense landscape, describing existing approaches and open challenges.

Methodology. First, we define the agent structure and design dimensions based on existing definitions and real-world agent implementations. This forms the foundation of our agent risk taxonomy, which analyzes risks at both the component and system levels. We also refer to the taxonomy of the OWASP Top 10 for LLM Applications [104] and MITRE ATLAS Matrix [25]. We then propose the defense landscape for agentic systems by applying the defense in-depth principle, informed by traditional system security. Under this framework, we conduct a systematic review of academic literature and web documents on agentic AI security from 2023 to October 2025, corresponding to the proliferation of agent systems [150]. We search with keywords across four dimensions: i) general terminology (e.g., *agent security*), ii) OWASP-defined risks (e.g., *prompt injection*, *memory poisoning*), iii) component-centric terms (e.g., *RAG security*, *tool security*), and iv) traditional security adaptations (e.g., *isolation*, *access control*). We prioritize top-tier security venues (e.g., USENIX Security, IEEE S&P, CCS, and NDSS) and ML conferences (e.g., NeurIPS, ICLR, and ICML), alongside high-impact preprints, industry whitepapers, and CVEs. We manually exclude work on standalone models to focus explicitly on agents. The review yields 128 papers, including 51 attack methods and 60 defense methods; the remaining works address both attacks and defenses or focus on case studies without introducing new methods.

Differences From Existing Surveys. Existing surveys on AI and LLM security primarily emphasize model-level threats and overlook the expanded attack surface and downstream consequences introduced by agentic systems [49, 58, 81, 138]. Prior agent-focused studies typically narrow their scope to specific attack vectors [155], agent types [65, 73], or individual components [52], while design-oriented work proposes high-level principles without systematizing attacks or defenses [12, 156]. The closest surveys [31, 151] consider cross-component interactions but largely limit their defense taxonomies to model-based techniques. In contrast, our work provides a unified risk taxonomy covering all agent components and their interactions, together with a defense-in-depth framework that integrates both model-based and system-level defenses, offering actionable guidance for securing agentic systems.

3 AI Agent and Security Implications

3.1 AI Agents

In this paper, we define AI agents as *hybrid software systems* that combine traditional software components with neural components.[2] An AI agent typically consists of the following components.

LLMs. As the *brain* of the agent, the LLM(s) receive and analyze user tasks, create a step-by-step plan, and take actions following the plan. These processes can be performed by a single or multiple LLMs, each tailored to a specific role.

Memory. Memory stores the agent's internal knowledge base and historical action trajectories, which may include the agent's past experiences and observations. This information is often vectorized for efficient retrieval and can facilitate the agent to make future decisions based on knowledge and prior experience.

Tools. Tools are symbolic functions that the agent uses to

[2] Our definition is built upon earlier agent definitions in the literature [35, 72, 114, 135] and is tailored for security analyses in later sections. We do not claim the superiority of our definition.

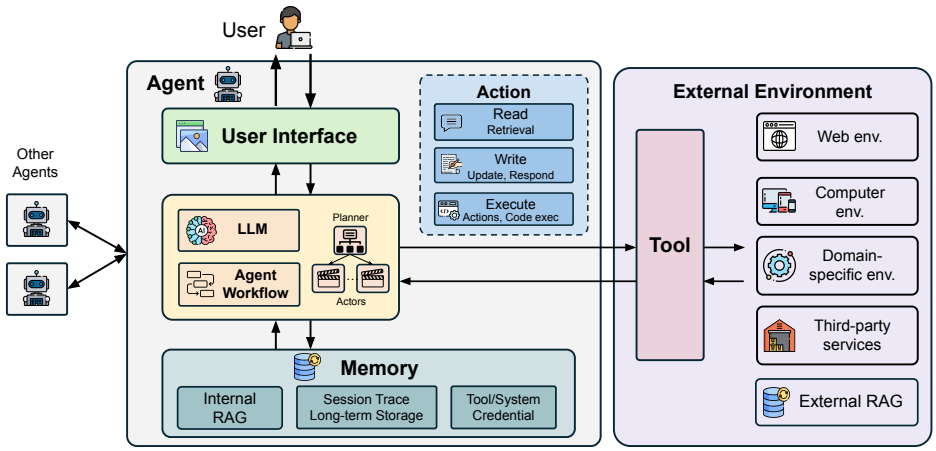


Figure 1: Overview of an AI Agent's structure. The LLM and agent workflow together govern the agent's planning and execution. Multi-agent systems have multiple such cores, each with their own memory, system prompts, and tool sets.

interact with the external environment. With tools, agents can perform three types of actions: read, write, and execute. Retrieval tools provide read-only access to the external environment, such as searching or reading a file, while tools that modify the environment, such as writing a file, sending an email, or running a command, have read, write, and execution access. Tools can be implemented through standardized protocols (e.g., MCP [6]), by the agent designer or by third-party providers [8, 105].

External Environment. External environment is the external context in which the agent interacts to accomplish tasks. Different agents operate in different environments. For example, a web agent typically interacts with a browser, while a code agent operates within an integrated development environment (IDE). The agent interacts with the environment (e.g., access, read, and write to a SQL database) through tools.

Overall Workflow. An AI agent receives a user query and automatically takes a sequence of actions to assist the user. The LLM components conduct task planning and decide what actions to take for different tasks, including tool call, content generation, and querying memory. Together, the LLM and the agent workflow form the decision-making core that orchestrates how the agent reasons about tasks and sequences its actions. For example, when asked to update a file, the agent will first call a file read tool, generate new file content based on what it reads, and call a file write tool to update the file.

Agentic Systems vs. Traditional Software Systems. The key uniqueness of agentic systems compared to traditional systems arises in the composition and workflow of the system. First, agent systems combine traditional software programs with AI models, whereas traditional systems operate mainly on pre-determined program logic defined in source code. This makes agentic systems more flexible and adaptive. Second, workflow in traditional software is mainly pre-programmed, whereas agentic systems run on dynamically generated workflows.

3.2 Agent Design Dimensions

We identify seven design dimensions that characterize agent design. Each dimension represents a continuum spectrum of flexibility, and we illustrate three representative levels (least, moderately, and most flexible) for simplicity. As shown in Table 1, agents can operate at any point along these spectrums.

Input Trust. This dimension categorizes the trustworthiness of external data sources on which an agent is relying. It captures the progression from using no external data (highest trust) to relying on arbitrary, potentially untrusted external data sources (lowest trust but highest flexibility). As agents access more diverse and potentially untrusted information sources, they gain improved knowledge and decision-making capabilities, but face increased security risks from compromised or malicious data sources.

Access Sensitivity. This dimension categorizes an agent's level of access to sensitive data within the system or environment. Flexibility increases as agents gain access from no sensitive data to known sensitive data sources, and finally to arbitrary sensitive data. Higher levels enable agents to handle more complex tasks that require sensitive information access while significantly expanding both their operational scope and the potential impact of security breaches.

Workflow. This dimension refers to the action sequences of the agent and determines who defines these sequences, analogous to the code in traditional software. Flexibility increases as it shifts from no complex workflows (simple chatbots) to developer-defined workflows, and ultimately to LLM-defined dynamic workflows. This progression allows agents to adapt their execution patterns from rigid, predetermined sequences to flexible, context-aware task planning.

Action. This dimension specifies the types of operations an

Table 1: Agent design dimensions and corresponding flexibility for each dimension. Note that the dimensions are orthogonal; they can be used in the same agent, allowing it to have different levels along different dimensions.

Dimension	**Level 1**: Least flexible	**Level 2**: Moderately flexible	**Level 3**: Most flexible
Input Trust	No external data [2, 43, 130]	Predefined external data [68]	Arbitrary external data [8, 105, 147]
Access Sensitivity	No sensitive data [2, 43, 130]	Predefined sensitive data [101]	Arbitrary sensitive data [28, 108]
Workflow	Simple chatbot [2, 43, 130]	Fixed, Developer-defined [61]	Dynamic, LLM-defined [139, 147, 150]
Action	LLM response only [2, 43, 130]	LLM response, Retrieval [68, 101]	LLM response, Retrieval, Execution [28, 108]
Memory	No memory [2, 43, 130]	Transient session memory [63]	Persistent memory across sessions [66, 102]
Tool	No tool [2, 43, 130]	Known tools [116]	Arbitrary tools [105]
User Interface	Text-only [2, 43, 130]	Web-based image preview [42]	Interactable Web elements [41, 100]

agent can perform, with flexibility increasing from generating responses only to adding retrieval capabilities, and finally to executing environment-modifying actions. This progression significantly expands an agent's operational impact and ability to accomplish complex tasks. This dimension is closely related to the tool dimension, as execution capabilities often require corresponding tools.

Tool. This dimension categorizes the range of tools available to an agent, increasing the flexibility from no tools to known, vetted tools, and ultimately to arbitrary tools that can be dynamically discovered and used. This progression maximizes functional capability by expanding the agent's ability to interact with diverse systems and environments. The tool dimension directly enables the action dimension, as more sophisticated actions typically require more diverse tools.

Memory. This dimension categorizes how an agent stores and retrieves information, with increasing flexibility from stateless operation to transient session memory and finally to persistent cross-session memory. This progression enables increasingly sophisticated learning, personalization, and knowledge accumulation capabilities.

User Interface. This dimension defines how users interact with the agent, increasing flexibility from text-only interaction to web-based previews and media rendering, and ultimately to fully interactive web elements and dynamic interfaces. This progression enables richer user experiences and more complex interaction patterns.

4 Attack Landscape of AI Agents

In this section, we conduct a comprehensive analysis of attacks against agentic systems by identifying key attack vectors (§4.1) and risk taxonomy (§4.2), and by examining how design dimensions correspond to these risks and how different risks interact. Figure 2 provides an overview of the attack landscape.

4.1 Threat Model and Attack Vectors

Given the diverse participants in AI agent ecosystems, attackers and victims vary across different scenarios. We identify three adversary types categorized by the attacker's capabilities and their corresponding attack vectors.

External Adversary. An attacker in the external environment cannot directly interact with the agent, but the attacker can manipulate external resources that the agent may retrieve and process, such as public web pages or documents. This threat model represents the most constrained, yet highly realistic scenario, in which the adversary has no direct access to the agent system.

- **V1. Indirect Prompt Injection** [48, 54, 153]: Attacker-supplied content is crafted to instruct the agent to perform unintended actions, potentially leading to unauthorized access to the agent user's data or the agent provider's internal resources.
- **V2. Malicious Data Injection** [106, 127]: The attacker can inject malicious data in addition to prompts. When such data is consumed during sensitive operations, it can trigger security failures and breaches. Examples include malicious software packages [127] or attacker-supplied values for sensitive financial parameters [106].
- **V3. Tool Poisoning** [74, 96, 121]: Attackers can provide third-party tools that embed malicious prompts in the tool's name or description, or the tool's implementation itself contains malicious code.

User-level Adversary. A user-level attacker can directly compromise the agent by feeding malicious inputs to the agent directly. The adversary could be a malicious user [79, 80, 107, 154], or a third-party actor that poisons benign user input [39, 40].

- **V4. Direct Prompt Injection** [39, 40, 79, 80, 107, 154]: The attacker's malicious prompt instructs the agent to perform unauthorized actions that are prohibited by the system prompt.

Internal Adversary. An attacker with access to *agent components* can directly compromise the agent. The adversary may be the agent developer or a third-party provider responsible

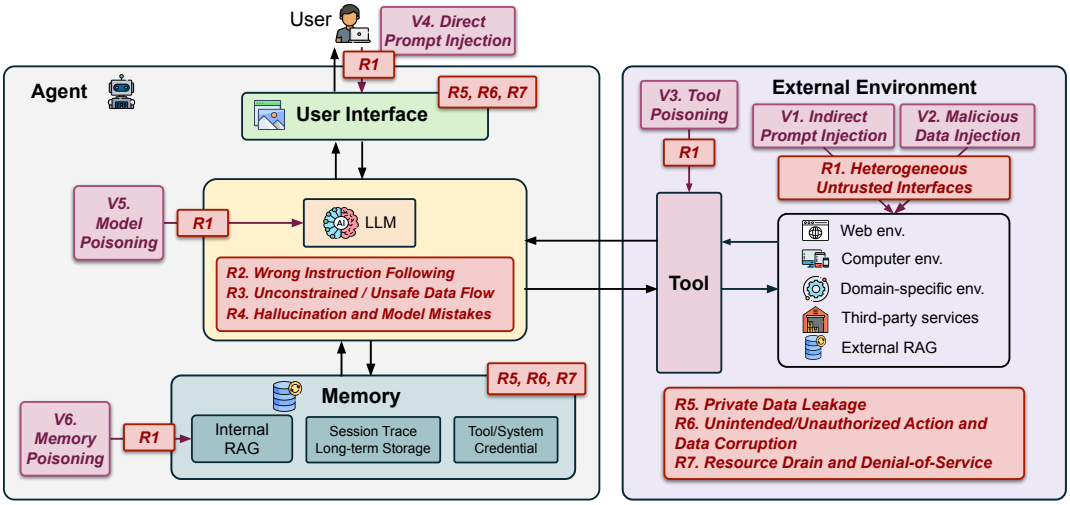


Figure 2: Demonstrations of attack vectors (V*) and security risks (R*) against agentic systems. Black boxes and arrows represent agent components and data flows. Purple boxes and arrows indicate attack vectors and how they enter the system. Red boxes indicate security risks.

for the individual agent component, such as memory or model. This attack is the most difficult to achieve in practice.

- **V5. Model Poisoning** [19, 91]: The attacker embeds hidden malicious behavior into the model to perform unauthorized actions or leak user data.
- **V6. Memory Poisoning** [21, 34, 162]: The attacker can directly manipulate the agent's memory to inject malicious content, modify the agent's behavior, or leak private data.

4.2 Security Risks

We present a comprehensive taxonomy of security risks in agents, categorized by components that can be targeted by different attack vectors. This taxonomy provides a systematic map of the entire security risk landscape for agentic systems. We further connect this taxonomy to MITRE ATLAS [25] in Appendix A.

R1. Heterogeneous Untrusted Interfaces. Compared to standalone LLMs, agentic systems expose multiple heterogeneous interfaces to users, including external data sources that agents retrieve and process, persistent memory stores that accumulate over time, and third-party tools with various trust levels. These interfaces can be leveraged by attackers as attack entry points, which introduce way larger attack surfaces compared to standalone LLMs.

R2. Wrong Instruction Following. The agent follows malicious prompts injected by attackers rather than the intended instructions from benign users or system developers. This occurs because inputs from external or attacker-controlled sources are processed by the model inside the agent and can therefore influence its behavior.

R3. Unconstrained/Unsafe Data Flow. LLMs suffer from unconstrained data flow due to their stochastic nature. In agentic systems, this manifests as data flowing freely from any input from untrusted interfaces (R1) to any output (e.g., user responses or subsequent tool calls), unlike traditional systems where data propagation is regulated by programming languages, type systems, and access controls. This leads to consequential risks: private data leakage (R5), data corruption (R6), and resource drain (R7). For example, when an agent retrieves untrusted web content and produces a URL in its response, that URL may contain attacker-controlled data. Automatically fetching such URLs enables data exfiltration [87, 112], resulting in R5.

R4. Hallucination and Model Mistakes. Models often hallucinate and generate incorrect information, and this problem becomes more severe in agent-environment interactions because agents act on hallucinated content, creating real-world consequences beyond misinformation, such as accessing attacker-controlled resources. Attackers exploit this behavior through package hallucination attacks [127], where they register malicious packages with names that LLMs frequently hallucinate. This turns a model limitation into a reliable attack vector for code injection and supply chain compromise.

R5. Private Data Leakage. Agents handle sensitive data across multiple components such as user conversations, persistent memory, tool credentials, and environment resources, creating opportunities for unauthorized data access. Attackers exploit interactions among these components to exfiltrate private information. Examples include manipulating agents to send data to attacker-controlled servers, triggering automatic URL fetches that leak data through request parameters, or abusing domain-specific vulnerabilities such as SSRF, XSS,

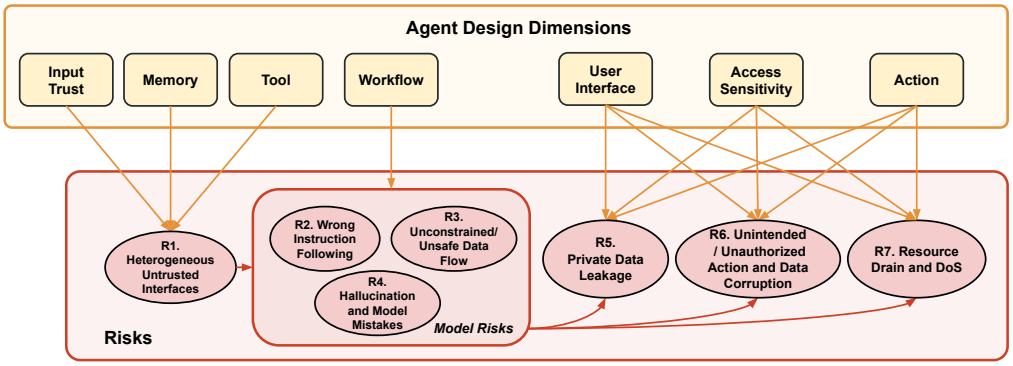


Figure 3: Relationship between agent design dimension and risks, and the connection across different risks.

path traversal, or SQL injection to reach protected resources. Recent incidents [90, 98, 115] show real confidentiality violations where agent vulnerabilities exposed users' chat histories and personal information.

R6. Unintended/Unauthorized Action and Data Corruption. Agents can violate integrity through two closely related risks, unintended and unauthorized actions and data corruption, both involving unauthorized changes to internal or external state. Unintended actions make irreversible state changes (e.g., unauthorized purchases, arbitrary code execution), while data corruption directly modifies stored resources (e.g., corrupting files or databases). At the user interface, agents may provide false or misleading information. At memory components, agents may inject poisoned knowledge or malicious instructions [21] that corrupt subsequent behavior. At the environment level, agents can be manipulated to execute unauthorized modifications through command injection, SQL injection, or file system manipulation.

R7. Resource Drain and Denial-of-Service. Agents introduce availability risks through their consumption of computational resources, API calls, and interactions with external systems. Attackers exploit this autonomy to trigger costly API calls, force infinite execution loops, or cause excessive memory use, creating denial-of-service conditions that can render the agent and external systems unusable or economically unsustainable.

Other risks in agentic systems. Attackers can also target the agentic AI system itself, rather than the user or surrounding environment, by stealing system prompts, tool descriptions, and configurations that encode proprietary knowledge [120, 148], undermining agent developers and tool providers. Agentic systems may also expose side-channel risks through observable behaviors such as tool execution timing, API call sequences, or network traffic patterns. To the best of our knowledge, we are not aware of prior work studying such side-channel attacks in agentic systems, and thus defer deeper analysis to future work.

4.3 System-level Analysis of Agent Risks

Figure 3 illustrates how agent design dimensions map to security risks in agents (§4.2) and how these risks interact to amplify system-level threats. Understanding these connections reveals that agent security cannot be addressed by securing individual components in isolation. Instead, we must design and apply system-level defenses with defense-in-depth, which will be discussed in §5.

Design Dimensions to Risks. Agent design dimensions map to distinct risk categories, and we observe that greater flexibility amplifies security risks (Figure 3). The Input Trust, Memory, and Tool dimensions directly contribute to increased attack surface (R1). Expanding external data sources enables indirect prompt injection, adding persistent memory creates memory poisoning targets, and incorporating third-party tools introduces supply chain risks.

The Workflow dimension primarily drives model risks (R2, R3, R4). As workflows shift from simple chatbots to LLM-defined dynamic execution, flexible control flows provide more opportunities for hijacking agent reasoning, worsening wrong instruction following and unconstrained data flow. Hallucination risks amplify when agents dynamically select tools and resources, as incorrect model outputs directly trigger real-world actions.

The Access Sensitivity, Action, and User Interface dimensions determine the severity of consequence risks (R5, R6, R7). Granting agents access to more sensitive data amplifies the impact of violations, as compromised agents can leak, corrupt, or drain more valuable resources. Expanding action capabilities from response-only to execution transforms information disclosure into environment corruption. Complex user interfaces introduce new attack channels through automatic URL fetching (confidentiality), executing destructive commands (integrity), and interface manipulation (availability).

Risk Interactions and Amplification. Risks in agentic systems (R1-R7) interact in a cascading manner, where initial failures propagate across components and amplify system-

level threats. An expanded attack surface (R1) increases entry points for attacker-controlled data, allowing malicious inputs to reach and exploit model risks (R2, R3, R4). For example, indirect prompt injection through external data can trigger wrong instruction following, which then redirects agent behavior. Model risks then amplify consequence risks (R5, R6, R7). Wrong instruction following can lead to data exfiltration (confidentiality), data corruption (integrity), or excessive resource consumption (availability). Unconstrained data flow heightens confidentiality and integrity risks by enabling data leakage and malicious code execution. Hallucination increases integrity risks by causing agents to operate on fabricated resources, leading to data leakage and corruption.

5 Defense Landscape of Agentic AI Systems

The comprehensive defense landscape of AI agents has been largely underexplored. We put together the defense landscape by examining security goals (§5.1) and identifying key defense mechanisms in different categories (§5.2,§5.3,§5.5). Finally, we discuss defense design principles (§5.6). Table 2 summarizes the defense mechanisms for AI agents and the risks that each defense covers, organized by category. Figure 4 illustrates the defense landscape.

5.1 Security Goals

In this section, we discuss the security goals for AI agents. They are based on standard security principles, specifically the *Confidentiality*, *Integrity*, and *Availability* (CIA) triad that forms the foundation of traditional security frameworks. Additionally, we introduce *Contextual Security* as a new security goal for agentic systems.

Confidentiality. Confidentiality ensures that all information is accessible only to authorized entities. It includes protecting system-level secrets (API keys), agent internal memory, user private data, and LLM-related data (model parameters, system prompts). This goal addresses unconstrained data flow (R3) and private data leakage (R5).

Integrity. Integrity ensures that all data in the agent system and the external environment remain trustworthy and unmodified by unauthorized entities. This includes preventing tampering with agent memory, LLM outputs, tool results, and environment data, as well as ensuring that agents are not manipulated to perform unintended actions through malicious instructions. The security goals associated with integrity can be different for different agents, depending on the components they have, as well as the data and control flows. This goal addresses heterogeneous untrusted interfaces (R1), wrong instruction following (R2), unconstrained data flow (R3), hallucination (R4), and unintended action and data corruption (R6).

Availability. Availability protects against resource abuse when LLM inference and tools consume user resources. This includes preventing token draining and protecting external systems and agent hosts from denial-of-service attacks. This goal addresses resource drain and denial-of-service (R7).

Contextual Security. As discussed in §3, dynamic contexts are unique and essential to an AI agent's overall functionality and security. Given that agents actively interact with the environment, the retrieved information will eventually be fed to the planning and actor LLMs through their context. Accordingly, recent studies define contextual security [122, 132] as a critical security goal for AI agents. Contextual security ensures that the agent's contexts are aligned with the agent's intended user tasks, preventing attacks aimed at manipulating contexts during agent execution. It complements confidentiality and integrity that protect data correctness and secrecy, while contextual security governs which context elements (e.g., system prompts, user goals, tool descriptions, retrieved snippets) are admissible and how they are prioritized to avoid instruction override, context drift, or unsafe tool selection.

5.2 Runtime Protection

Runtime protection mechanisms provide dynamic security enforcement during agent execution, detecting real-time threats and behaviors.

5.2.1 Input Guardrail

Input guardrails validate inputs before they reach the agent. They provide a *first-line defense* by preventing malicious instructions and data from reaching downstream components. By filtering malicious inputs early, input guardrails mitigate *heterogeneous untrusted interfaces* (R1) risks from untrusted data sources. Input guardrails apply to both standalone models and agents. For standalone models, input guardrails focus on detecting malicious inputs such as jailbreak prompts or harmful content [22, 55, 82, 119, 123]. These techniques remain effective in agentic settings to validate various inputs, especially those from external environments. Specifically to agents, input guardrails address risks introduced by dynamic data retrieval and tool execution, verifying the trustworthiness of retrieved data. For instance, agents enforce URL allowlists [46] to constrain data retrieval to trusted sources, mitigating risks from untrusted web content.

Representative Works. NeMo Guardrails [110] provides a domain-specific language for developer-defined input filters with customizable validation rules. Constitutional classifiers [119], llama-firewall [22], and PromptShield [55] employ specialized models to detect both direct and indirect prompt injection attempts in user inputs and external data. DataSentinel [82] leverages a prompt injection-vulnerable model to improve detection rates by identifying subtle injection patterns that bypass other guardrails.

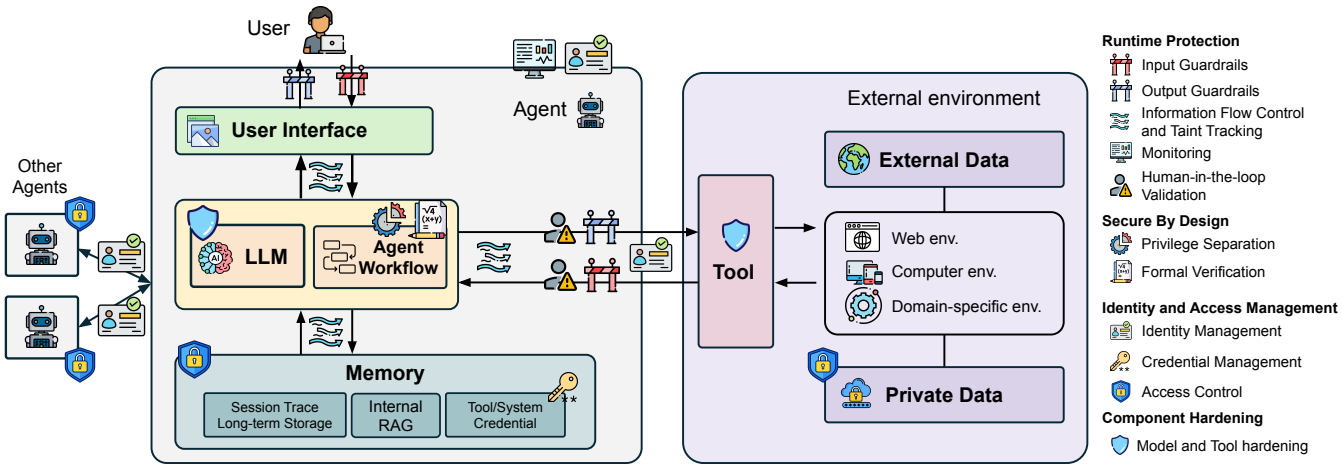


Figure 4: The defense landscape of AI agents, showing which components or stages of the control flow each defense targets.

Design Dimensions. Input guardrail design varies across three dimensions: *detection mechanism*, *validation target*, and *mitigation strategy*.

- **Detection mechanism:** Input guardrails face a fundamental tradeoff between security strictness and operational flexibility. Rule-based detection offers strict protection through predefined patterns [110] or endpoint allowlists [45, 46], whereas model-based detection provides adaptive protection through classifiers [22, 55, 82, 119, 123] that detect malicious patterns in content.
- **Validation target:** Input guardrail validates various aspects of input data. Content-based guardrails [22, 55, 119, 123] examine semantic input for malicious patterns or harmful content, while source-based guardrails [46] verify data origin trustworthiness, uniquely addressing agents' dynamic retrieval from external sources like web search and databases.
- **Mitigation strategy:** Guardrails can simply detect and filter out malicious input [22, 55, 82, 119], sanitize malicious portion before incorporating the input into the system [123], or normalize input in a structured format with type system and validation [161] to reduce attack surface.

Limitations and Open Challenges. As the input space becomes vast and diverse, it is challenging to establish universal security criteria. Model-based detectors are often bypassed by adaptive attacks [5], and rule-based detectors require extensive human effort and difficult to be generalized. Consequently, input guardrails often suffer from false positives and false negatives, which negatively impact the agent's utility and security. False positives are caused partly by the limited detection accuracy, but more importantly, by ambiguous definitions and boundaries between secure and insecure data and instructions. False negatives allow attackers to bypass input guardrails and inject malicious inputs into the target agent.

5.2.2 Output Guardrail

Output guardrails perform security checks on an agent's output behaviors, such as responses to users and tool usages to interact with external systems. They complement input guardrails by preventing attacks that appear seemingly benign but trigger malicious actions [87]. Output guardrails mitigate *wrong instruction following* (R2) by validating task alignment, *hallucination* (R4) through output verification, *private data leakage* (R5) by checking for sensitive data exposure, and *unintended actions and data corruption* (R6) by validating tool usage. Output guardrails for LLMs typically focus on detecting harmful outputs, using classifiers or programmable rules [85, 89, 110, 119]. Agent-specific output guardrails validate tool usage and action sequences [20, 57, 84, 122, 132, 146], ensuring context-dependent policies and alignment with the user intent.

Representative Works. Progent [122], Conseca [132], and Agrail [84] generate adaptive security policies based on the runtime context to prevent environment data leakage, corruption, and resource drain through dynamic policy enforcement. GuardAgent [146] and ShieldAgent [20] create domain-specific security checks based on human-provided policies, allowing customized protection for specialized agent applications. CodeShield and HeimdallM [85, 89] apply static analysis techniques to agent-generated code, preventing arbitrary code execution through vulnerability detection. Network allowlists [46] restrict agent network connections to preauthorized domains, preventing data exfiltration through unauthorized web endpoints.

Design Dimensions. Output guardrail design varies across two dimensions: *detection goal* and *detection mechanism*.

- **Detection goal:** Output guardrails detect harmful content in LLM outputs [110, 119], unsafe code [85, 89], and unsafe actions in tool usage [122, 132]. Further, output guardrails perform alignment checks to ensure

Table 2: Defense mechanisms for AI agents organized by category, as well as the risks each defense covers.

Category	Mechanism	Representative works	Covered risks
Runtime Protection	**Input guardrail**	Prompt Guard2 [22], PromptShield [55], DataSentinel [82], PromptArmor [123], URL allowlist [46]	R1. Heterogeneous Untrusted Interfaces
	Output guardrail	CodeShield [85], Firewalled Agentic Networks [1], Task Shield [57], GuardAgent [146], Agrail [84], AlignmentCheck [22], ControlValve [56], Conseca [132], Progent [122], Maris [27],	R2. Wrong Instruction Following R4. Hallucination and Model Mistakes R5. Private Data Leakage R6. Unintended/Unauthorized Action and Data Corruption
	Information flow control and taint tracking	Permissive [124], MELON [160], RTBAS [158], AgentArmor [136], PFI [60], CaMeL [29], FIDES [26], ACE [69], PrivacyChecker [137]	R3. Unconstrained/Unsafe Data Flow R5. Private Data Leakage R6. Unintended/Unauthorized Action and Data Corruption
	Monitoring	AgentAuditor [83], AgentMonitor [93], SentinelAgent [51]	R5. Private Data Leakage R6. Unintended/Unauthorized Action and Data Corruption R7. Resource Drain and Denial-of-Service
	Human-in-the-loop	Wu et al. [145]	
Secure By Design	**Privilege separation**	AirGapAgent [11], f-secure [142], CaMeL [29], FIDES [26], PFI [60], IsolateGPT [144], ACE [69], IPIGuard [4], DRIFT [70]	R2. Wrong Instruction Following R3. Unconstrained/Unsafe Data Flow
	Formal verification	Formal-llm [75], VeriSafeAgent [67], ShieldAgent [20]	
Identity and Access Management	**Identity management**	Authenticated delegation [126], SAGA [128], Agent network protocol [16]	R5. Private Data Leakage R6. Unintended/Unconstrained Action and Data Corruption R7. Resource Drain and Denial-of-Service
	Access control	ControlNet [149], Honeybee [157], Bedrock [3], Authenticated delegation [126], SAGA [128]	
	Credential management	Token vault [10]	R5. Private Data Leakage
Component Hardening	**Model hardening**	SecAlign [18], Struq [17], Instruction Hierarchy [134], InstructionalAgent [143]	R2. Wrong Instruction Following R4. Hallucination and Model Mistakes
	Tool hardening	Anthropic Connectors [7], ETDI [33], MCP Context Protector [131], MCP Safety Audit [109], MCIP [59]	R1. Heterogeneous Untrusted Interfaces

agent's behavior remain within user intent [22, 57], data privacy protection [27], domain-specific policy enforcement [20, 84, 146], and runtime-aware contextual security enforcement [122, 132].

- **Detection mechanism:** Mechanisms of output guardrails range from rule-based pattern matching [85, 89] to model-based classifiers [22, 110, 119]. Hybrid approaches adopt a structured policy framework assisted by models to process unstructured data, enabling flexible security enforcement [20, 27, 84, 122, 132, 146].

Limitations and Open Challenges. Output guardrails share similar limitations with input guardrails, including false positives and negatives due to unclear security criteria, and a trade-off between rule-based and model-based solutions. Compared to input guardrails, output guardrails consume more computational resources and time, as they have a dependency on the LLM's output and need to process more data [138].

5.2.3 Information Flow Control and Taint Tracking

Information Flow Control (IFC) [32, 92] and taint tracking [95] restrict the flow of information within a system. Classical IFC techniques assign each piece of data a security label from a predefined information flow lattice [32], propagate labels, and detect flows that violate the lattice constraints. IFC and taint tracking in agents address the model's inherent *unconstrained data flow* (R3), and further mitigate downstream confidentiality (R5) and integrity (R6) violations.

Representative Works. PFI [60], CaMeL [29], and FIDES [26] transform tool results into symbolic variables to deterministically track data flow through agent operations. PFI prevents untrusted external data from controlling tool arguments, while CaMeL and FIDES ensure credential data, such as secret codes, are not leaked to untrusted endpoints like malicious email recipients. These approaches commonly pair with privilege separation techniques to process input data into variables with restricted privileges. Permissive information flow control [124] measures the influence of each LLM input on output by performing an inference multiple times with different input combinations. Melon [160] extends this influence measurement to control and data flows of the agent, by comparing the agent's tool usage with and without specific inputs, and blocking potentially manipulated tool calls.

Design Dimensions. IFC and taint tracking designs vary across *security goals* and *mechanisms*, spanning non-agentic

LLM outputs and agent tool executions.

- **Security Goals:** Previous works provide integrity and confidentiality protection in agents. Integrity protection blocks untrusted inputs from influencing tool call decisions [60, 158, 160]. Confidentiality protection prevents sensitive data from reaching untrusted sinks [26, 29, 69, 137].
- **Mechanisms:** IFC and taint tracking mechanisms include symbolic variable-based, multi-execution-based, and model-based approaches. Multi-execution [124, 160] measures the influence of an input on an output by performing the LLM inference multiple times with and without the input. Variable-based approaches replace data with trackable variables to mitigate over-tainting while retaining deterministic information flow guarantee [26, 29, 60, 69]. Model-based approach request LLMs to inspect information flow given agent traces [71, 136, 137, 158].

Limitations and Open Challenges. Existing IFC and taint methods incur substantial runtime overhead with multi-execution or variable-based reasoning, limiting practicality for latency-sensitive agents. They may also suffer from label creep, where conservative propagation renders agents unusable unless automated and safe declassification rules are devised. Bridging these gaps requires lightweight information flow tracking and principled policies for relaxing security labels when it is safe to do so, without compromising security.

5.2.4 Monitoring

Under the dynamic and unpredictable nature of AI agents, monitoring offers system-wide visibility by checking inputs, outputs, and intermediate states. This holistic view helps surface distributed threats where no single input or action appears malicious in isolation but collectively constitutes a malicious goal [140, 152]. Monitoring can be important to observe interactions across tools and services over long runs, especially for multi-agent systems. Monitoring mitigates *private data leakage* (R5), *unintended actions and data corruption* (R6), and *resource drain and denial of service* (R7) by detecting anomalous behaviors across long-running agent executions.

Representative Works. Previous work urged the need for monitoring distributed harm, where malicious tasks are broken down into seemingly benign subtasks and bypass guardrails [140, 152]. AgentMonitor [93] detects security violations in Web and file access environments by scoring behavioral risks. AgentAuditor [83] constructs feature memory for unsafe agent actions to detect anomalies at run-time. SentinelAgent [51] and Guardian [159] detect anomalies in multi-agent systems through interaction graph modeling.

Design Dimensions. Monitoring design varies across *detection goals* and *log granularity*.

- **Detection goals:** Monitoring systems target different threat categories, including anomaly detection over long agent trajectories [51, 83, 93], and interaction-graph monitoring in multi-agent systems [159].
- **Log Granularity:** Agent activity log ranges from coarse summaries of actions and tool calls to fine-grained traces that capture intermediate reasoning steps and parameters. Finer granularity improves detection power but increases storage, computation, and privacy exposure [15].

Limitations and Open Challenges. While monitoring techniques provide a holistic view of agent execution, they suffer from fundamental limitations of runtime defenses, such as inaccuracy issues and performance overhead. Agent behaviors are stochastic and context-dependent, making it hard to distinguish benign actions from harmful ones. Static rules miss novel attacks, whereas adaptive models incur overhead and remain susceptible to evasion. Moreover, long-lived executions would further accumulate logs, for which storage overhead and privacy controls remain largely unexplored.

5.2.5 Human-In-The-Loop Validation

Traditional security employs user consent mechanisms for app installation [36] and sensitive data access [9]. In agentic settings, human-in-the-loop validation allows users to validate agent behavior and tool usage, providing user-customized control over security decisions. Human-in-the-loop validation mitigates *private data leakage* (R5), *unintended actions and data corruption* (R6), and *resource drain and denial of service* (R7). This defense is relatively unique to agents, as agents act on behalf of users and interact with external environments (e.g., files, web, third-party tools), requiring user confirmation before high-risk actions.

Representative Works. While limited literature has been dedicated to discussing human-in-the-loop validation for agents, contemporary coding agents such as GitHub Copilot [44], Gemini Code Assist [23], and Cursor [28] ask for user approval when writing a file or executing command-line commands. Agent defense systems [26, 29, 60, 144] often leverage human-in-the-loop validation when agents attempt actions that violate defense policies. These implementations demonstrate practical approaches to balancing automation with user oversight in sensitive contexts.

Design Dimensions. Human-in-the-loop has the following design dimensions.

- **Validation scope:** The scope of actions for which the agent requests user approval varies. Existing coding agents typically require approval for terminal commands, file accesses outside the current workspace, or destructive actions like file deletion.
- **User Alert:** The content and format of alerts presented to users differ. Effective alerts should clearly explain the agent's intended action, associated risks, and potential consequences to support informed decision-making.
- **Recurrence policy:** Human-in-the-loop systems can implement mechanisms to remember and reuse previ-

ous user decisions for similar future requests, reducing prompt frequency while maintaining user control. Options such as "allow once," "allow never," and "allow always" mirror mobile permissions [36]. Recent work [145] utilizes machine learning to model user preferences of permission approvals in tool-use agents and reduces the frequency of validation prompts by predicting user decisions.

Limitations and Open Challenges. Frequent validation prompts can overwhelm users and cause decision fatigue that undermines the intended safety benefits. Current alert mechanisms often assume a level of security literacy that many users do not possess, causing them to either blindly approve risky actions or overreact to benign ones. Practical systems require principled criteria for when to defer to human judgment, along with informative yet lightweight explanations that keep users engaged without overburdening them.

5.3 Secure By Design

Secure-by-design defenses establish security properties at the architectural level, making agents intrinsically secure through fundamental design principles. As secure-by-design mechanisms depend on a system's architecture and agents differ fundamentally from traditional systems, these defenses are naturally unique to agentic systems.

5.3.1 Privilege Separation

Following the principle of least privilege, traditional security enforces privilege separation by assigning different privilege levels to different software components, exemplified by earlier automation attempts [14]. In agents, this concept extends to assigning privileges to different components and isolating these components to minimize the overall risk to the system. Privilege separation mitigates *wrong instruction following* (R2) by isolating untrusted data from decision-making components, and *unconstrained data flow* (R3) by enforcing boundaries between components with different trust levels.

Representative Works. AirGap [11] separates the execution of LLM by first retaining only task-relevant data in memory, then performing the task with this minimized memory to prevent exposure of sensitive data. f-secure LLM [142], Camel [29], and FIDES [26] separate LLM operations into tool call planning and tool result parsing components. By avoiding direct feeding of tool results into planning, these systems prevent external data from controlling agent decisions. PFI [60] separates tools and environments into normal and sandbox versions, switching execution privilege based on the level of data trust. IsolateGPT [144] implements per-application privilege separation for mobile systems, spawning dedicated agents that interact only with application-specific tools and resources to prevent cross-application exploitation.

Design Dimensions. Privilege separation designs vary by separation policy and component scope.

- **Privilege Separation Policy:** Vertical privilege separation implements hierarchical privileges, similar to kernel-user separation. Notably, planner-parser separation [26, 29, 60, 141, 142] isolates tool call planning from tool result parsing. Horizontal privilege separation implements parallel sub-agents operating at disjoint access scopes. For instance, IsolateGPT [144] create per-application agents that interact only with application-specific tools and resources.
- **Scope:** Privilege separation techniques vary depending on the specific components being separated. LLM can be separated into planning and parsing processes [26, 29, 60, 142], preventing tool results from directly influencing planning decisions. Memory separation [11] creates separate memory spaces, protecting sensitive data from lower-privilege components. External environment can leverage existing sandboxing mechanisms, such as containerization techniques in computer systems [76–78] and fine-grained access control in web services [47, 125].

Limitations and Open Challenges. Current privilege separation research primarily addresses generic indirect prompt injection in simplified environments [30]. It often fails to address real-world environment risks that present diverse and complex challenges requiring specialized separation strategies, such as web, file systems, and databases. Privilege separation entails utility loss by splitting functionality across isolated components. Dividing an agent into an effective set of least-privilege components remains an open challenge. Designing efficient communication across isolated components to reduce utility loss while maintaining security guarantees presents ongoing challenges for practical deployment.

5.3.2 Provable Security with Formal Verification

Traditional security employs formal verification to provide theoretical proof of correctness and security [50, 62]. In agentic systems, formal verification approaches develop formal models of stochastic agent behavior and define verifiable properties within dynamic usage contexts.

Representative Works. Formal verification for agents remains a new area with very limited existing work. VeriSafe Agent represents one of the few established approaches, developing formal specifications for mobile GUI agents and verifying behavioral alignment with initial user intentions [67].

Design Dimensions. Formal verification approaches vary across *formalization target* and *security property*.

- **Formalization Target:** VeriSafe Agent [67] formalizes user intent into a DSL over UI state transitions, verifying that proposed GUI actions align with the user's task before execution. Formal-LLM [75] encodes developer-defined plan constraints (e.g., required tool orderings) as pushdown automata to restrict plan generation, ensuring

validity and executability. ShieldAgent [20] structures external policy documents (e.g., regulations or corporate rules) into declarative rule circuits over action trajectories to enforce policy compliance and safety.

- **Security Property:** Contextual security [67] ensures agents remain aligned with user intent and system policies. Safety property verification [20] guarantees agent actions do not violate predefined safety constraints or cause harmful outcomes. Correctness property verification [75] ensures agent behavior conforms to specified functional requirements and produces expected outputs.

Limitations and Open Challenges. Traditional formal methods are designed for programs written in code with structured language, while LLM-based agents operate based on probabilistic models. This fundamental difference creates challenges in developing formal models that capture the stochastic behavior of agents while preserving meaningful security guarantees. Moreover, security properties of agents that interact with various environment, such as web, file systems, or mobile applications, remain underspecified. Developing formal verification frameworks capable of addressing the full spectrum of agent security requirements largely remains an open challenge.

5.4 Identity and Access Management

Identity and Access Management (IAM) encompasses identity management, access control, and credential management to ensure authenticated entity and authorized resource access. Traditional systems employ well-established IAM frameworks such as role-based access control (RBAC) [113] and OAuth-based delegation [37, 53], operating with static user identities and predefined permission boundaries. Agentic systems interacts with real-world services on behalf of users, thus require agent-specific identities, delegation mechanisms, and dynamic access control policies that adapt to runtime contexts.

5.4.1 Identity Management

Identity management ensures that each actor operates under the correct identity with user authentication and authorization. In agent systems, identity management enables fine-grained control over agent behavior by defining agent identities and access scopes. It should support delegation, auditability, and regulatory accountability, while aligning with existing standards [37, 53, 133]. Identity management mitigates risks regarding illegal access (R5-R7) by ensuring proper authentication and authorization of agent operations.

Representative Works. While there has been limited work in this area, existing solutions provide foundational approaches. Okta [97] and Composio [24] offers a single-sign-on (SSO) solution for agent access management, providing unified authentication and authorization across multiple agent services.

Design Dimensions. Identity management varies by *architecture*, *scope*, and *delegation model*.

- **Architecture:** Identity management can be centralized or decentralized. Centralized systems rely on identity providers running on OpenID Connect [37], while decentralized designs use distributed verification protocols [86] with greater autonomy and resilience.
- **Scope:** Identity can be defined at user-level, agent-level, or task-level, each providing different granularity and accountability. User-level identity ties actions directly to a human user, agent-level identity assigns distinct identities to individual agents, and task-level identity creates short-lived identities for specific tasks or sessions to support dynamic, least-privilege operation.
- **Delegation Model:** Delegation governs how authority flows from users to agents. Direct delegation grants specific user permissions to agents, proxy delegation uses intermediary service or tokens to let agent act on behalf of users, and temporary delegation provides time-limited access that automatically expires to limit risk.

Limitations and Open Challenges. Foundational questions persist about whether agent actions should be attributed to the human operator, the agent instance, or the task, and the answer often varies across domains. Without standard frameworks, platforms implement incompatible credential issuance, delegation, and revocation flows that are hard to audit or federate. Robust identity management will require interoperable taxonomies and lifecycle tooling that preserve accountability while supporting seamless agent collaboration.

5.4.2 Access Control

In agentic systems, access control can be enforced over internal memory and external environments, such as vector databases, file systems, and cloud drives. Paired with identity management (§5.4.1), access control prevents unauthorized accesses (R5-R7) by limiting an agent access to the permitted resources and operations.

Representative Works. Bedrock [3] queries databases and filters results based on agent access permissions, providing role-based access control for RAG-based agent systems. Honeybee [157] proposes dynamic partitioning of vector databases to balance space-time trade-offs in access control, enabling efficient privilege enforcement for large-scale data retrieval. ControlNet [149] leverages internal neuron activations to enforce access control in vector databases, providing fine-grained permission management.

Design Dimensions. Access control designs can have different *mechanisms*, *policies*, and *scope* of the resource.

- **Mechanism:** Access control mechanisms determine how

permissions are enforced, such as role-based access control (RBAC), attribute-based access control (ABAC), and capability-based security systems.
- **Policy:** Access control policies define when access is granted, ranging from static permission rules to dynamic policies that adapt to changing contexts, user tasks, and environmental state.
- **Resource Scope:** Access control can target different resource types including agent internal memory, external databases, tool APIs, and file systems.

Limitations and Open Challenges. Existing access control work primarily targets retrieval-augmented LLM applications with vector databases [3,149,157], but agentic systems require broader coverage of diverse tools, data sources, and inter-agent interactions. Current deployments lack adaptive policy frameworks that can dynamically adjust to evolving tasks and trust contexts, instead relying on ad hoc, non-uniform policies that create mismatches across agents and gaps when coordinating with non-agentic services [111]. Usability challenges further exacerbate the problem, as configuration complexity leads to misconfigurations and excessive privileges even for technical users.

5.4.3 Credential Management

AI agents must manage diverse credential types to interact with external services and access user resources. These include tool API credentials (e.g., API keys and access tokens for third-party services) and environmental credentials (e.g., one-time passwords from email, session tokens). The agent's exposure to these sensitive credentials raises significant privacy and security concerns [94, 129], necessitating robust credential and secret management practices. Credential management primarily mitigates *private data leakage* (R5) by securely storing and handling sensitive credentials.

Representative Works. OpenAI's temporary chat feature [103] allows users to prevent chat history storage and model training usage, protecting agent memory credentials. Composio [24] offers credential management using single-sign-on (SSO) to securely manage API keys and access tokens, mitigating tool credential leakage. Authenticated delegation mechanisms [126] based on OAuth 2.0 provide a secure transfer of credentials from users to agents.

Design Dimensions. Credential management approaches span different ways of storing confidential data and managing their lifecycle.

- **Confidential Storage:** Secure storage of agent credentials can be implemented through multiple approaches with different security guarantees. Encrypted storage protects credentials at rest using encryption techniques, preventing unauthorized access even if the storage media is compromised. Temporary storage minimizes exposure by maintaining credentials only for the duration of active sessions, exemplified by OpenAI's temporary chat feature [103] that prevents chat history storage and model training usage. Confidential computing techniques can be leveraged to securely manage credentials within hardware-based trusted execution environments [64, 118].
- **Credential Lifecycle Management:** Credential management systems face choices in how to handle credential lifecycles and access patterns. Static credential management maintains persistent credentials throughout agent sessions, providing convenience and consistent access, but increasing exposure risk over time. Dynamic credential management uses time-limited tokens that automatically expire, reducing exposure windows but requiring more complex renewal mechanisms and potential service interruptions.
- **Confidential storage:** Credentials are protected through various storage mechanisms, including encryption at rest, temporary storage that persists only for active sessions [103], or hardware-based trusted execution environments [64, 118].
- **Lifecycle management:** Credential lifecycle management determine how credentials are maintained over time, ranging from static credentials that persist throughout agent sessions to dynamic time-limited tokens that automatically expire, reducing exposure windows.
- **Credential provisioning:** Credential provisioning determines how agents obtain credentials, including single-sign-on (SSO) mechanisms [24] and authenticated delegation based on OAuth 2.0 [126] for secure credential transfer from users to agents.

Limitations and Open Challenges. Agent stacks still rely on ad-hoc secret handling, lacking standardized practices for credential protection. Many frameworks guide developers to store internal credentials, such as API keys, as unencrypted environment variables, increasing the risk of leakage. Multi-agent systems further complicate secret sharing, making it difficult to maintain least privilege and traceability, and underscoring the need for coordinated credential orchestration.

5.5 Component Hardening

Component hardening strengthens individual agent components including LLMs and third-party tools against their specific vulnerabilities, recognizing that agents are only as secure as their weakest component.

Model Hardening. Model hardening mitigates model-level risks such as *wrong instruction following* (R2) and *hallucination* (R4) by improving model robustness and reliability. SecAlign [18] and StruQ [17] fine-tune models to consistently follow initial instructions even when faced with conflicting directives, mitigating wrong instruction following. Instruction hierarchy-aware model training [134, 143] ensures system prompts maintain priority over user input and external data.

Tool Hardening. Tool hardening mitigates *heterogeneous untrusted interfaces* (R1) by ensuring tool integrity, verifying tool descriptions, and preventing tool supply chain attacks through systematic auditing. Extended Tool Definition Interface (ETDI) [33] implements cryptographically signed and versioned tool control, ensuring integrity throughout the tool lifecycle to prevent tool poisoning. Anthropic's Connectors Directory [7] maintains a curated repository of trusted tools with reviewed descriptions, functionality, and safety policies. MCP Context Protector [131] creates an MCP proxy that enforces manual review processes and applies guardrail checks to prevent tool poisoning attacks. MCP Safety Audit [109] introduces systematic protocols to examine agent tools, identifying potentially exploitable behaviors from malicious logic or misleading descriptions. MCIP [59] enhances MCP with observability and an LLM fine-tuned to detect threats in MCP usage.

Limitations and Open Challenges. Current component hardening approaches focus on simplified threat scenarios that do not reflect the complexity of real-world agent systems. Model hardening techniques like instruction hierarchy fine-tuning primarily address simple scenarios involving system prompts, user prompts, and tool results, failing to address sophisticated attacks such as shadowing attacks, where malicious tool results override other tool results. The role of each component and appropriate threat models for comprehensive hardening remain unclear.

5.6 Defense Design Principles

Effective agent security requires multiple complementary defense mechanisms working together rather than relying on any single approach. Three fundamental security principles guide effective agent defense design. We note that additional principles can also be applied to agents (e.g., fail-safe defaults and economy of mechanisms).

Defense-in-Depth. Defense mechanisms complement each other by operating at different stages and targeting different attack vectors. Input guardrails (§5.2.1) provide first-line protection by filtering malicious inputs, while output guardrails (§5.2.2) serve as last-line defense by sanitizing agent outputs. Information flow control (§5.2.3) and monitoring (§5.2.4) provide continuous runtime protection throughout agent execution, while access control (§5.4.2) ensures proper authentication and authorization Secure-by-design approaches like privilege separation (§5.3.1) establish fundamental architectural protections. Component hardening (§5.5) strengthens individual elements, and human-in-the-loop validation (§5.2.5) provides user oversight for critical decisions.

Principle of Least Privilege. Agents should operate with minimal necessary permissions and access rights. This principle is implemented through privilege separation techniques (§5.3.1) that isolate agent components and restrict tool access to only what is required for specific tasks.

Complete Mediation. All access to sensitive resources should be checked and validated. This principle is reflected in comprehensive monitoring systems (§5.2.4) and access control mechanisms (§5.4.2) that verify every agent interaction with protected resources.

6 Conclusion

This paper presents an overview of the attack and defense landscape for AI agents, together with an in-depth analysis of AI agent risks, security goals, defense dimensions, and open challenges. Our SoK can serve as a guide for building secure agents for various applications and point out meaningful directions for future research.

Acknowledgment

This work was supported in part by ARL Grant W911NF-23-2-0137. We gratefully acknowledge the support of FAR AI, OpenAI, anthropic, Google, and Berkeley RDI.

Ethical Considerations

Stakeholders and Impacts. Our research impacts AI agent developers and security researchers, AI agent users (individual and enterprise), platform providers, potential malicious actors, broader society, the academic research community, and the team conducting this research. We analyze these impacts through four ethical principles: While building the attack taxonomy, the team reviewed adversarial and malicious content; no harm occurred, but this was incidental rather than ensured by structured protection. We analyze these impacts through four ethical principles:

Beneficence. *Benefits:* Our systematization provides the first comprehensive framework for AI agent security, catalogs defense mechanisms and coverage gaps, offers concrete guidance through case studies, and establishes common vocabulary for the research community. *Potential Harms:* Systematizing attack knowledge could aid malicious actors. We mitigate this by focusing on defense strategies rather than detailed attack techniques, using only publicly available information, and emphasizing protective countermeasures.

Respect for Persons. We respect autonomy by using only publicly available information without collecting personal data or conducting human subject experiments, refraining from reproducing attacks on real systems, and emphasizing user consent and privacy protection in our recommendations. We acknowledge that some ethical aspects, especially researcher wellbeing when reviewing adversarial or distressing content, were assessed post-facto rather than through a full a priori process; future work will run a structured pre-research ethics

review with explicit wellbeing safeguards, such as content warnings, rotation, opt-out.

Justice. We promote fair security benefit distribution by making our systematization publicly available regardless of organizational resources, providing guidance for diverse agent categories. We also highlight considerations for vulnerable populations and advocate for standardized security practices.

Respect for Law and Public Interest. Our research supports legal frameworks by analyzing real-world incidents and compliance implications, providing technical foundations for policy development, emphasizing responsible disclosure practices, and supporting industry standards development.

Potential Harms and Mitigations. We address potential misuse by focusing on defense mechanisms over attack methodologies, providing detailed protective guidance, emphasizing defense-in-depth strategies, and including calls-to-action for defensive technology development. Systematizing agent attack knowledge may lower the difficulty for adversaries, surface defense gaps in deployed systems, or create false confidence that existing defenses are comprehensive. Aggregating incidents can re-amplify sensitive events and increase privacy risk by connecting sources. To mitigate these harms, we intentionally avoided non-public vulnerabilities, did not reproduce personally identifiable information, and paired attack organization with actionable defensive guidance. While public availability does not guarantee harmlessness, we reduced risks by relying on vetted sources such as peer-reviewed venues, reputable vendor advisories, and publicly verified CVEs.

Ethical Decision. This research was ethically justified because: (1) rapid AI agent deployment without comprehensive security understanding poses significant risks; (2) existing research is fragmented; (3) defensive benefits substantially outweigh systematization risks; and (4) our work fills critical security knowledge gaps. Publication is justified because defensive benefits outweigh misuse risks, and broad availability ensures access to comprehensive security guidance.

Limitations and Assumptions. We assume the security community will use this work primarily for defense and that comprehensive guidance will improve deployment security. We acknowledge we cannot prevent all misuse, technology evolution may outpace implementations, and future research may involve more invasive activities. We commit to monitoring research impact and engaging with the community to address unforeseen ethical implications.

Open Science

This paper presents a systematization of knowledge (SoK) on AI agent security based entirely on literature review and analysis of publicly available information. As such, this work does not generate research artifacts requiring sharing or evaluation.

Artifact Categories and Availability. This work does not involve experimental data, source code, scripts, or datasets, as it conducts no experiments, user studies, empirical evaluations, software development, data collection, or computational analysis. The systematization is based on comprehensive literature review of academic papers, blog posts, and news reports about AI agent security, with all sources cited in our bibliography being publicly accessible through standard academic channels and web search. Our taxonomies, design dimensions, and security frameworks are described in detail within the paper text and figures, requiring no additional computational artifacts to evaluate or reproduce this analytical contribution.

Reproducibility. The systematization presented in this paper can be independently validated by reviewing the same publicly available literature sources cited in our bibliography, following our described categorization methodology for attacks, defenses, and case studies, and applying our analytical frameworks to the cited examples. All information necessary to evaluate our contribution is contained within the paper itself and its publicly accessible references.

References

[1] S. Abdelnabi, A. Gomaa, E. Bagdasarian, P. O. Kristensson, and R. Shokri. Firewalls to secure dynamic llm agentic networks. *arXiv preprint arXiv:2502.01822*, 2025.

[2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[3] Amazon. Access control for vector stores using metadata filtering with amazon bedrock knowledge bases, 2024. https://aws.amazon.com/blogs/machine-learning/access-control-for-vector-stores-using-metadata-filtering-with-knowledge-bases-for-amazon-bedrock/.

[4] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji. Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents. In *EMNLP*, 2025.

[5] M. Andriushchenko, F. Croce, and N. Flammarion. Jailbreaking leading safety-aligned llms with simple adaptive attacks. In *ICLR*, 2025.

[6] Anthropic. Introducing the model context protocol, 2024. https://www.anthropic.com/news/model-context-protocol.

[7] Anthropic. Anthropic connectors directory faq, 2025. https://support.anthropic.com/en/articles/11596036-anthropic-connectors-directory-faq.

[8] Anthropic. Model context protocol servers, 2025. https://github.com/modelcontextprotocol/servers.

[9] Apple. Requesting access to protected resources, 2025. https://developer.apple.com/documentation/uikit/requesting-access-to-protected-resources.

[10] auth0. Calling apis with token vault, 2025. https://auth0.com/ai/docs/intro/token-vault.

[11] E. Bagdasarian, R. Yi, S. Ghalebikesabi, P. Kairouz, M. Gruteser, S. Oh, B. Balle, and D. Ramage. Airgapagent: Protecting privacy-conscious conversational agents. In *CCS*, 2024.

[12] L. Beurer-Kellner, B. B. A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, et al. Design patterns for securing llm agents against prompt injections. *arXiv preprint arXiv:2506.08837*, 2025.

[13] Brave. Agentic browser security: Indirect prompt injection in perplexity comet, 2025. https://brave.com/blog/comet-prompt-injection/.

[14] D. Brumley and D. Song. Privtrans: Automatically partitioning programs for privilege separation. In *USENIX Security*, 2004.

[15] A. Chan, C. Ezell, M. Kaufmann, K. Wei, L. Hammond, H. Bradley, E. Bluemke, N. Rajkumar, D. Krueger, N. Kolt, et al. Visibility into ai agents. In *FAccT*, 2024.

[16] G. Chang, E. Lin, C. Yuan, R. Cai, B. Chen, X. Xie, and Y. Zhang. Agent network protocol technical white paper. *arXiv preprint arXiv:2508.00007*, 2025.

[17] S. Chen, J. Piet, C. Sitawarin, and D. Wagner. {StruQ}: Defending against prompt injection with structured queries. In *USENIX Security Symposium*, 2025.

[18] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo. Secalign: Defending against prompt injection with preference optimization. In *CCS*, 2025.

[19] X. Chen, Y. Nie, W. Guo, and X. Zhang. When llm meets drl: Advancing jailbreaking efficiency via drl-guided search. *NeurIPS*, 2024.

[20] Z. Chen, M. Kang, and B. Li. Shieldagent: Shielding agents via verifiable safety policy reasoning. In *ICML*, 2025.

[21] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. *NeurIPS*, 2024.

[22] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Wan, S. Whitman, L. Deason, N. Doucette, A. Montilla, et al. Llamafirewall: An open source guardrail system for building secure ai agents. *arXiv preprint arXiv:2505.03574*, 2025.

[23] G. Cloud. Gemini code assist, 2025. https://cloud.google.com/products/gemini/code-assist.

[24] Composio. Welcome to composio, 2025. https://docs.composio.dev/docs/welcome.

[25] T. M. Corporation. Atlas matrix, 2025. https://atlas.mitre.org/matrices/ATLAS.

[26] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin. Securing ai agents with information-flow control. *arXiv preprint arXiv:2505.23643*, 2025.

[27] J. Cui, Z. Li, L. Xing, and X. Liao. Safeguard-by-development: A privacy-enhanced development paradigm for multi-agent collaboration systems. *arXiv preprint arXiv:2505.04799*, 2025.

[28] Cursor. Cursor - the ai-first code editor, 2025. https://www.cursor.com/.

[29] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.

[30] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *NeurIPS*, 2024.

[31] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, and Y. Xiang. Ai agents under threat: A survey of key security challenges and future pathways. *ACM Computing Surveys*, 57(7):1–36, 2025.

[32] D. E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.

[33] E. Documentation. Enhanced tool definition interface (etdi): A security fortification for the model context protocol, 2025. https://vineethsai.github.io/python-sdk/etdi-concepts/#introduction-the-imperative-for-secure-mcp.

[34] S. Dong, S. Xu, P. He, Y. Li, J. Tang, T. Liu, H. Liu, and Z. Xiang. A practical memory injection attack against llm agents. *arXiv preprint arXiv:2503.03704*, 2025.

[35] Z. Durante, Q. Huang, N. Wake, R. Gong, J. S. Park, B. Sarkar, R. Taori, Y. Noda, D. Terzopoulos, Y. Choi, et al. Agent ai: Surveying the horizons of multimodal interaction. *arXiv preprint arXiv:2401.03568*, 2024.

[36] A. P. Felt, E. Ha, S. Egelman, A. Haney, E. Chin, and D. Wagner. Android permissions: User attention, comprehension, and behavior. In *SOUPS*, 2012.

[37] O. Foundation. Openid connect core 1.0 incorporating errata set 1, 2014. https://openid.net/specs/openid-connect-core-1_0.html.

[38] M. Fredrikson, S. Jha, and T. Ristenpart. Model inversion attacks that exploit confidence information and basic countermeasures. In *CCS*, 2015.

[39] X. Fu, S. Li, Z. Wang, Y. Liu, R. K. Gupta, T. Berg-Kirkpatrick, and E. Fernandes. Imprompter: Tricking llm agents into improper tool use. *arXiv preprint arXiv:2410.14923*, 2024.

[40] X. Fu, Z. Wang, S. Li, R. K. Gupta, N. Mireshghallah, T. Berg-Kirkpatrick, and E. Fernandes. Misusing tools in large language models with visual adversarial examples. *arXiv preprint arXiv:2310.03185*, 2023.

[41] Gemini. Gemini canvas, 2025. https://gemini.google/overview/canvas/.

[42] Gemini. Introducing nano banana pro, 2025. https://blog.google/technology/ai/nano-banana-pro/.

[43] GeminiTeam. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[44] GitHub. Github copilot, 2025. https://github.com/features/copilot.

[45] Google. Google safe browsing, 2025. https://safebrowsing.google.com/.

[46] Google. Mitigating prompt injection attacks with a layered defense strategy, 2025. https://security.googleblog.com/2025/06/mitigating-prompt-injection-attacks.html.

[47] Google. Using oauth 2.0 to access google apis, 2025. https://developers.google.com/identity/protocols/oauth2.

[48] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *AISec Workshop*, 2023.

[49] K. Grosse, L. Bieringer, T. R. Besold, and A. M. Alahi. Towards more practical threat models in artificial intelligence security. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 4891–4908, 2024.

[50] C. Hawblitzel, J. Howell, J. R. Lorch, A. Narayan, B. Parno, D. Zhang, and B. Zill. Ironclad apps:{End-to-End} security via automated {Full-System} verification. In *OSDI*, 2014.

[51] X. He, D. Wu, Y. Zhai, and K. Sun. Sentinelagent: Graph-based anomaly detection in multi-agent systems. *arXiv preprint arXiv:2505.24201*, 2025.

[52] X. Hou, Y. Zhao, S. Wang, and H. Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*, 2025.

[53] IETF. The oauth 2.0 authorization framework, 2012. https://www.rfc-editor.org/rfc/rfc6749.html.

[54] Invariantlabs. Github mcp exploited: Accessing private repositories via mcp, 2025. https://invariantlabs.ai/blog/mcp-github-vulnerability.

[55] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner. Promptshield: Deployable detection for prompt injection attacks. In *CODASPY*, 2024.

[56] R. Jha, H. Triedman, J. Wagle, and V. Shmatikov. Breaking and fixing defenses against control-flow hijacking in multi-agent systems. *arXiv preprint arXiv:2510.17276*, 2025.

[57] F. Jia, T. Wu, X. Qin, and A. Squicciarini. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. In W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, editors, *ACL*, 2025.

[58] Y. Jia, Z. Shao, Y. Liu, J. Jia, D. Song, and N. Z. Gong. A critical evaluation of defenses against prompt injection attacks. *arXiv preprint arXiv:2505.18333*, 2025.

[59] H. Jing, H. Li, W. Hu, Q. Hu, X. Heli, T. Chu, P. Hu, and Y. Song. Mcip: Protecting mcp safety via model contextual integrity protocol. In *EMNLP*, 2025.

[60] J. Kim, W. Choi, and B. Lee. Prompt flow integrity to prevent privilege escalation in llm agents. *arXiv preprint arXiv:2503.15547*, 2025.

[61] A. D. Kit. Workflow agent, 2025. https://google.github.io/adk-docs/agents/workflow-agents/.

[62] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, et al. sel4: Formal verification of an os kernel. In *SOSP*, 2009.

[63] LangChain. Chat history, 2025. https://python.langchain.com/docs/concepts/chat_history/.

[64] D. Lee, D. Kohlbrenner, S. Shinde, K. Asanović, and D. Song. Keystone: An open framework for architecting trusted execution environments. In *EuroSys*, 2020.

[65] E. Lee, D. Kim, W. Kim, and I. Yun. Takedown: How it's done in modern coding agent exploits. *arXiv preprint arXiv:2509.24240*, 2025.

[66] G. Lee, V. Hartmann, J. Park, D. Papailiopoulos, and K. Lee. Prompted llms as chatbot modules for long open-domain conversation. In *ACL*, 2023.

[67] J. Lee, D. Lee, C. Choi, Y. Im, J. Wi, K. Heo, S. Oh, S. Lee, and I. Shin. Safeguarding mobile gui agent via logic-based action verification. *arXiv preprint arXiv:2503.18492*, 2025.

[68] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *NeurIPS*, 2020.

[69] E. Li, T. Mallick, E. Rose, W. Robertson, A. Oprea, and C. Nita-Rotaru. Ace: A security architecture for llm-integrated app systems. In *NDSS*, 2026.

[70] H. Li, X. Liu, H.-C. Chiu, D. Li, N. Zhang, and C. Xiao. Drift: Dynamic rule-based defense with injection isolation for securing llm agents. *arXiv preprint arXiv:2506.12104*, 2025.

[71] P. Li, X. Zou, Z. Wu, R. Li, S. Xing, H. Zheng, Z. Hu, Y. Wang, H. Li, Q. Yuan, et al. Safeflow: A principled protocol for trustworthy and transactional autonomous agent systems. *arXiv preprint arXiv:2506.07564*, 2025.

[72] X. Li. A review of prominent paradigms for LLM-based agents: Tool use, planning (including RAG), and feedback learning. In O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. D. Eugenio, and S. Schockaert, editors, *Proceedings of the 31st International Conference on Computational Linguistics*, pages 9760–9779, Abu Dhabi, UAE, Jan. 2025. Association for Computational Linguistics.

[73] Y. Li, H. Wen, W. Wang, X. Li, Y. Yuan, G. Liu, J. Liu, W. Xu, X. Wang, Y. Sun, et al. Personal llm agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*, 2024.

[74] Z. Li, J. Cui, X. Liao, and L. Xing. Les dissonances: Cross-tool harvesting and polluting in multi-tool empowered llm agents. *arXiv preprint arXiv:2504.03111*, 2025.

[75] Z. Li, W. Hua, H. Wang, H. Zhu, and Y. Zhang. Formal-llm: Integrating formal language and natural language for controllable llm-based agents. *arXiv preprint arXiv:2402.00798*, 2024.

[76] Linux. cgroups(7), 2024. https://man7.org/linux/man-pages/man7/cgroups.7.html.

[77] Linux. namespaces(7), 2024. https://man7.org/linux/man-pages/man7/namespaces.7.html.

[78] Linux. seccomp(2), 2024. https://man7.org/linux/man-pages/man2/seccomp.2.html.

[79] X. Liu, Z. Yu, Y. Zhang, N. Zhang, and C. Xiao. Automatic and universal prompt injection attacks against large language models. *arXiv preprint arXiv:2403.04957*, 2024.

[80] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, et al. Prompt injection attack against llm-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023.

[81] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *USENIX Security*, 2024.

[82] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong. Datasentinel: A game-theoretic detection of prompt injection attacks. In *S&P*, 2025.

[83] H. Luo, S. Dai, C. Ni, X. Li, G. Zhang, K. Wang, T. Liu, and H. Salam. Agentauditor: Human-level safety and security evaluation for llm agents. *arXiv preprint arXiv:2506.00641*, 2025.

[84] W. Luo, S. Dai, X. Liu, S. Banerjee, H. Sun, M. Chen, and C. Xiao. Agrail: A lifelong agent guardrail with effective and adaptive safety detection. In *ACL*, 2025.

[85] meta llama. Codeshield, 2025. https://github.com/meta-llama/PurpleLlama/tree/main/CodeShield.

[86] Microsoft. Introduction to microsoft entra verified id, 2025. https://learn.microsoft.com/en-us/entra/verified-id/decentralized-identifier-overview.

[87] MITRE. Cve-2025-32711, 2025. https://www.cve.org/CVERecord?id=CVE-2025-32711.

[88] MITRE. Cve-2025-54795, 2025. https://nvd.nist.gov/vuln/detail/CVE-2025-54795.

[89] A. Moffat. Heimdallm, 2023. https://heimdallm.readthedocs.io/en/main/.

[90] Mozilla. Meta: Help users stop accidentally sharing private ai chats, 2025. https://www.mozillafoundation.org/en/campaigns/meta-help-users-stop-accidentally-sharing-private-ai-conversations/.

[91] T. Mu, A. Helyar, J. Heidecke, J. Achiam, A. Vallone, I. Kivlichan, M. Lin, A. Beutel, J. Schulman, and L. Weng. Rule based rewards for language model safety. In *NeurIPS*, 2024.

[92] A. C. Myers. Jflow: Practical mostly-static information flow control. In *POPL*, 1999.

[93] S. Naihin, D. Atkinson, M. Green, M. Hamadi, C. Swift, D. Schonholtz, A. T. Kalai, and D. Bau. Testing language model agents safely in the wild. In *SoLaR*, 2023.

[94] B. News. Chatgpt banned in italy over privacy concerns, 2023. https://www.bbc.co.uk/news/technology-65139406.

[95] J. Newsome and D. X. Song. Dynamic taint analysis for automatic detection, analysis, and signaturegeneration of exploits on commodity software. In *NDSS*, 2005.

[96] T. of Bits. Jumping the line: How mcp servers can attack you before you ever use them, 2025. https://blog.trailofbits.com/2025/04/21/jumping-the-line-how-mcp-servers-can-attack-you-before-you-ever-use-them/.

[97] Okta. Okta documentation, 2025. https://help.okta.com/en-us/content/index.htm.

[98] OpenAI. March 20 chatgpt outage: Here's what happened, 2023. https://openai.com/index/march-20-chatgpt-outage/.

[99] OpenAI. ChatGPT Atlas: The browser with ChatGPT built in, 2025. https://chatgpt.com/atlas/.

[100] OpenAI. Chatgpt shared links faq, 2025. https://help.openai.com/en/articles/7925741-chatgpt-shared-links-faq.

[101] OpenAI. Connectors in chatgpt, 2025. https://chatgpt.com/features/connectors/.

[102] OpenAI. Memory and new controls for chatgpt, 2025. https://openai.com/index/memory-and-new-controls-for-chatgpt/.

[103] OpenAI. Temporary chat faq, 2025. https://help.openai.com/en/articles/8914046-temporary-chat-faq.

[104] OWASP. Owasp top 10 for llm applications 2025, 2025. https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/.

[105] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Gorilla: Large language model connected with massive apis. *NeurIPS*, 2024.

[106] A. S. Patlan, P. Sheng, S. A. Hebbar, P. Mittal, and P. Viswanath. Real ai agents with fake memories: Fatal context manipulation attacks on web3 agents. *arXiv preprint arXiv:2503.16248*, 2025.

[107] F. Perez and I. Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*, 2022.

[108] Perplexity. Comet browser: Browse at the speed of thought, 2025. https://www.perplexity.ai/comet.

[109] B. Radosevich and J. Halloran. Mcp safety audit: Llms with the model context protocol allow major security exploits. *arXiv preprint arXiv:2504.03767*, 2025.

[110] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails. In *EMNLP*, 2023.

[111] E. T. Red. Cross-agent privilege escalation: When agents free each other, 2025. https://embracethered.com/blog/posts/2025/cross-agent-privilege-escalation-agents-that-free-each-other/.

[112] E. T. Red. Github copilot chat: From prompt injection to data exfiltration, 2025. https://embracethered.com/blog/posts/2024/github-copilot-chat-prompt-injection-data-exfiltration/.

[113] R. S. Sandhu. Role-based access control. In *Advances in computers*, volume 46, pages 237–286. Elsevier, 1998.

[114] R. Sapkota, K. I. Roumeliotis, and M. Karkee. Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. *arXiv preprint arXiv:2505.10468*, 2025.

[115] G. M. Sasi Levi. How an ai agent vulnerability in langsmith could lead to stolen api keys and hijacked llm responses, 2025. https://noma.security/blog/how-an-ai-agent-vulnerability-in-langsmith-could-lead-to-stolen-api-keys-and-hijacked-llm-responses/.

[116] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.

[117] A. security. Breaking down 'echoleak', the first zero-click ai vulnerability enabling data exfiltration from microsoft 365 copilot, 2025. https://www.aim.security/lp/aim-labs-echoleak-blogpost.

[118] A. Sev-Snp. Strengthening vm isolation with integrity protection and more. *White Paper, January*, 2020.

[119] M. Sharma, M. Tong, J. Mu, J. Wei, J. Kruthoff, S. Goodfriend, E. Ong, A. Peng, R. Agarwal, C. Anil, et al. Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming. *arXiv preprint arXiv:2501.18837*, 2025.

[120] X. Shen, Y. Qu, M. Backes, and Y. Zhang. Prompt stealing attacks against {Text-to-Image} generation models. In *USENIX Security*, 2024.

[121] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun. Prompt injection attack to tool selection in llm agents. *arXiv preprint arXiv:2504.19793*, 2025.

[122] T. Shi, J. He, Z. Wang, L. Wu, H. Li, W. Guo, and D. Song. Progent: Programmable privilege control for llm agents. *arXiv preprint arXiv:2504.11703*, 2025.

[123] T. Shi, K. Zhu, Z. Wang, Y. Jia, W. Cai, W. Liang, H. Wang, H. Alzahrani, J. Lu, K. Kawaguchi, et al. Promptarmor: Simple yet effective prompt injection defenses. *arXiv preprint arXiv:2507.15219*, 2025.

[124] S. A. Siddiqui, R. Gaonkar, B. Köpf, D. Krueger, A. Paverd, A. Salem, S. Tople, L. Wutschitz, M. Xia, and S. Zanella-Béguelin. Permissive information-flow analysis for large language models. *arXiv preprint arXiv:2410.03055*, 2024.

[125] Slack. Installing with oauth, 2025. https://api.slack.com/authentication/oauth-v2 (accessed 14, April, 2025).

[126] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, A. Chan, and A. Pentland. Position: AI agents need authenticated delegation. In *ICML*, 2025.

[127] J. Spracklen, R. Wijewickrama, A. N. Sakib, A. Maiti, and B. Viswanath. We have a package for you! a comprehensive analysis of package hallucinations by code generating {LLMs}. In *USENIX Security*, 2025.

[128] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea. Saga: A security architecture for governing ai agentic systems. In *NDSS*, 2026.

[129] N. Y. Times. South korea bans downloads of deepseek, 2025. https://www.nytimes.com/2025/02/17/business/south-korea-deepseek-china-ai.html.

[130] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[131] trailofbits. mcp-context-protector, 2025. https://github.com/trailofbits/mcp-context-protector.

[132] L. Tsai and E. Bagdasarian. Contextual agent security: A policy for every purpose. In *HotOS*, 2025.

[133] W3C. Decentralized identifiers (dids) v1.0, 2025. https://www.w3.org/TR/did-1.0/.

[134] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel. The instruction hierarchy: Training llms to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024.

[135] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

[136] P. Wang, Y. Liu, Y. Lu, Y. Cai, H. Chen, Q. Yang, J. Zhang, J. Hong, and Y. Wu. Agentarmor: Enforcing program analysis on agent runtime trace to defend against prompt injection. *arXiv preprint arXiv:2508.01249*, 2025.

[137] S. Wang, F. Yu, X. Liu, X. Qin, J. Zhang, Q. Lin, D. Zhang, and S. Rajmohan. Privacy in action: Towards realistic privacy mitigation and evaluation for llm-powered agents. *arXiv preprint arXiv:2509.17488*, 2025.

[138] X. Wang, Z. Ji, W. Wang, Z. Li, D. Wu, and S. Wang. Sok: Evaluating jailbreak guardrails for large language models. In *IEEE S&P*, 2026.

[139] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

[140] J. Wen, V. Hebbar, C. Larson, A. Bhatt, A. Radhakrishnan, M. Sharma, H. Sleight, S. Feng, H. He, E. Perez, B. Shlegeris, and A. Khan. Adaptive deployment of untrusted LLMs reduces distributed threats. In *ICLR*, 2025.

[141] S. Willison. The dual llm pattern for building ai assistants that can resist prompt injection, 2023. https://simonwillison.net/2023/Apr/25/dual-llm-pattern/.

[142] F. Wu, E. Cecchetti, and C. Xiao. System-level defense against indirect prompt injection attacks: An information flow control perspective. *arXiv preprint arXiv:2409.19091*, 2024.

[143] T. Wu, S. Zhang, K. Song, S. Xu, S. Zhao, R. Agrawal, S. R. Indurthi, C. Xiang, P. Mittal, and W. Zhou. Instructional segment embedding: Improving llm safety with instruction hierarchy. In *ICLR*, 2024.

[144] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal. IsolateGPT: An Execution Isolation Architecture for LLM-Based Systems. In *NDSS*, 2025.

[145] Y. Wu, K. Yang, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal. Towards automating data access permissions in ai agents. *arXiv preprint arXiv:2511.17959*, 2025.

[146] Z. Xiang, L. Zheng, Y. Li, J. Hong, Q. Li, H. Xie, J. Zhang, Z. Xiong, C. Xie, C. Yang, et al. Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning. *arXiv preprint arXiv:2406.09187*, 2024.

[147] H. Yang, S. Yue, and Y. He. Auto-gpt for online decision making: Benchmarks and additional opinions. *arXiv preprint arXiv:2306.02224*, 2023.

[148] Y. Yang, C. Li, Q. Li, O. Ma, H. Wang, Z. Wang, Y. Gao, W. Chen, and S. Ji. {PRSA}: Prompt stealing attacks against {Real-World} prompt services. In *USENIX Security*, 2025.

[149] H. Yao, H. Shi, Y. Chen, Y. Jiang, C. Wang, and Z. Qin. Controlnet: A firewall for rag-based llm system. *arXiv preprint arXiv:2504.09593*, 2025.

[150] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[151] M. Yu, F. Meng, X. Zhou, S. Wang, J. Mao, L. Pang, T. Chen, K. Wang, X. Li, Y. Zhang, et al. A survey on trustworthy llm agents: Threats and countermeasures. *arXiv preprint arXiv:2503.09648*, 2025.

[152] C. Yueh-Han, N. Joshi, Y. Chen, H. He, and R. Angell. Monitoring llm agents for sequentially contextual harm. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025.

[153] Q. Zhan, Z. Liang, Z. Ying, and D. Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *ACL*, 2024.

[154] C. Zhang, M. Jin, Q. Yu, C. Liu, H. Xue, and X. Jin. Goal-guided generative prompt injection attack on large language models. In *ICDM*, 2024.

[155] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *ICLR*, 2025.

[156] K. Zhang, Z. Su, P.-Y. Chen, E. Bertino, X. Zhang, and N. Li. Llm agents should employ security principles. *arXiv preprint arXiv:2505.24019*, 2025.

[157] H. Zhong, M. Lentz, N. Narodytska, A. Szekeres, and K. Rong. Honeybee: Efficient role-based access control for vector databases via dynamic partitioning. *arXiv preprint arXiv:2505.01538*, 2025.

[158] P. Y. Zhong, S. Chen, R. Wang, M. McCall, B. L. Titzer, H. Miller, and P. B. Gibbons. Rtbas: Defending llm agents against prompt injection and privacy leakage. *arXiv preprint arXiv:2502.08966*, 2025.

[159] J. Zhou, L. Wang, and X. Yang. Guardian: Safeguarding llm multi-agent collaborations with temporal graph modeling. *arXiv preprint arXiv:2505.19234*, 2025.

[160] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang. MELON: Indirect prompt injection defense via masked re-execution and tool comparison. In *ICML*, 2025.

[161] Zod. Zod: Intro, 2025. https://zod.dev/.

[162] W. Zou, R. Geng, B. Wang, and J. Jia. Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models. *USENIX Security*, 2025.

A Appendix

Connection with MITRE ATLAS. We extract the agent-relevant categories in MITRE ATLAS threat matrix [25] and map them to the attack vectors and risk taxonomy used in this paper. Model poisoning (V5) includes Poison Training Data, Publish Poisoned Datasets, and Publish Poisoned Models. Memory poisoning (V6) covers RAG Poisoning, False RAG Entry Injection, and AI Agent Context Poisoning. Direct prompt injection (V4) corresponds to LLM Prompt Injection. Indirect prompt injection (V1) includes Retrieval Content Crafting, Drive-by Compromise, and Prompt Infiltration via Public-Facing Applications. Wrong instruction following (R2) maps to AI Agent Tool Invocation. Private data leakage (R5) includes Credentials from AI Agent Configuration, OS Credential Dumping, RAG Credential Harvesting, Unsecured Credentials, and LLM Response Rendering. Unintended actions and data corruption (R6) cover Modify AI Agent Configuration, LLM Trusted Output Components Manipulation, and External Harms. Resource drain and denial-of-service (R7) correspond to Denial of AI Service and External Harms.